

RAG_system

December 28, 2025

1 Config Settings Rag

```
[1]: # =====  
# CELL 1 - RAG CONFIG (Multi-Source Folder Ingestion)  
# =====  
  
from __future__ import annotations  
import os  
import re  
import json  
import hashlib  
from pathlib import Path  
from dataclasses import dataclass  
from typing import List, Dict, Optional  
  
# -----  
# 1) Project / Unit Settings  
# -----  
UNIT_CODE = "MBAI5004" # used for naming DB + outputs  
SOURCES_DIR = Path("inputs/sources") / UNIT_CODE # folder containing pdf/epub  
↳sources  
OUTPUT_ROOT = Path("data_output") / UNIT_CODE  
  
# Optional: unit outline path (used later to generate per-week topic queries)  
UNIT_OUTLINE_PATH = Path("MBAI5004 AI, Ethics and Governance 04112025.docx")  
  
# -----  
# 2) File Discovery Settings  
# -----  
ALLOWED_EXTS = {".pdf", ".epub"}  
PREFERRED_EXT_ORDER = [".epub", ".pdf"] # if both exist for same work_id, pick  
↳epub  
RECURSIVE = True  
  
# -----  
# 3) Vector DB Settings  
# -----
```

```

CHROMA_PERSIST_DIR = OUTPUT_ROOT / "chroma_db"
CHROMA_COLLECTION_NAME = f"{UNIT_CODE}_multi_source"

REBUILD_INDEX = False # True = wipe and rebuild; False = reuse if exists
    ↳(you'll implement append/upsert later)

# -----
# 4) Chunking Settings
# -----
CHUNK_SIZE = 900
CHUNK_OVERLAP = 120

# Optional advanced mode: parent/child (recommended once multi-source grows)
USE_PARENT_CHILD = True
PARENT_CHUNK_SIZE = 2200
CHILD_CHUNK_SIZE = 450
CHILD_CHUNK_OVERLAP = 80

# -----
# 5) Embeddings / Retrieval Settings
# -----
EMBEDDING_MODEL_OLLAMA = "nomic-embed-text" # example: your current Ollama
    ↳embedding model
TOP_K = 8
USE_HYBRID_RETRIEVAL = True # dense + BM25 ensemble (hybrid search)

# -----
# 6) Output Layout
# -----
TOC_DIR = OUTPUT_ROOT / "toc"
MANIFEST_PATH = OUTPUT_ROOT / "sources_manifest.json"
LOG_DIR = OUTPUT_ROOT / "logs"

for p in [OUTPUT_ROOT, TOC_DIR, LOG_DIR]:
    p.mkdir(parents=True, exist_ok=True)

# -----
# 7) Helpers (IDs + discovery)
# -----
def _normalize_work_id(path: Path) -> str:
    """
    Groups pdf/epub variants of the same work.
    Example: "Some Book (2019).pdf" and "Some Book (2019).epub" -> same work_id.
    """
    stem = path.stem.lower()
    stem = re.sub(r"\s+", " ", stem).strip()
    stem = re.sub(r"[\(\)\[\]\{\}]", "", stem)

```

```

stem = re.sub(r"[^a-z0-9_-]+", "", stem)
return stem[:140] # keep it short-ish

def _source_id_quick(path: Path) -> str:
    """
    Fast-ish stable ID: path + size + mtime.
    If you want stronger dedupe, switch to SHA256(file_bytes).
    """
    stat = path.stat()
    raw = f"{path.as_posix()}|{stat.st_size}|{int(stat.st_mtime)}".
    ↪ encode("utf-8")
    return hashlib.sha1(raw).hexdigest() # short stable id

def discover_sources() -> List[Dict]:
    files = []
    it = SOURCES_DIR.rglob("*") if RECURSIVE else SOURCES_DIR.glob("*")
    for fp in it:
        if fp.is_file() and fp.suffix.lower() in ALLOWED_EXTS:
            files.append(fp)

    # Group by work_id (so pdf/epub duplicates are handled)
    grouped: Dict[str, List[Path]] = {}
    for fp in files:
        grouped.setdefault(_normalize_work_id(fp), []).append(fp)

    selected = []
    for work_id, variants in grouped.items():
        # choose preferred ext if multiple
        variants_sorted = sorted(
            variants,
            key=lambda p: PREFERRED_EXT_ORDER.index(p.suffix.lower())
            if p.suffix.lower() in PREFERRED_EXT_ORDER else 999
        )
        chosen = variants_sorted[0]
        selected.append({
            "work_id": work_id,
            "source_id": _source_id_quick(chosen),
            "path": str(chosen),
            "ext": chosen.suffix.lower(),
            "all_variants": [str(v) for v in variants_sorted],
        })
    return sorted(selected, key=lambda x: x["path"])

SOURCES = discover_sources()
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump({"unit": UNIT_CODE, "sources": SOURCES}, f, indent=2, ↪
    ↪ ensure_ascii=False)

```

```
print(f"Discovered {len(SOURCES)} source works in: {SOURCES_DIR}")
print(f"Manifest written to: {MANIFEST_PATH}")
```

Discovered 4 source works in: inputs/sources/MBAI5004

Manifest written to: data_output/MBAI5004/sources_manifest.json

2 Helper functions

```
[2]: # Cell 10: Configuration and Scoping for Content Generation (Corrected)

import os
import json
import logging

# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
↳ %(message)s')
logger = logging.getLogger(__name__)

# --- Global Test Overrides (for easy testing) ---
TEST_OVERRIDE_WEEKS = None
TEST_OVERRIDE_FLOW_ID = None
TEST_OVERRIDE_SESSIONS_PER_WEEK = None
TEST_OVERRIDE_DISTRIBUTION_STRATEGY = None

def process_and_load_configurations():
    """
    PHASE 1: Loads configurations, calculates a PRELIMINARY time-based slide_
↳ budget,
    and saves the result as 'processed_settings.json' for the Planning Agent.
    """
    print_header("Phase 1: Configuration and Scoping Process", char="-")

    # --- Load all input files ---
    logger.info("Loading all necessary configuration and data files...")
    try:
        os.makedirs(CONFIG_DIR, exist_ok=True)
        with open(PARSED_UO_JSON_PATH, 'r', encoding='utf-8') as f:
            unit_outline = json.load(f)
        with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
            book_toc = json.load(f)
```

```

        with open(SETTINGS_DECK_PATH, 'r', encoding='utf-8') as f:
↳ settings_deck = json.load(f)
        # with open(TEACHING_FLOWS_PATH, 'r', encoding='utf-8') as f:
↳ teaching_flows = json.load(f)
        logger.info("All files loaded successfully.")
    except FileNotFoundError as e:
        logger.error(f"FATAL: A required configuration file was not found: {e}")
        return None

    # --- Pre-process and Refine Settings ---
    logger.info("Pre-processing settings_deck for definitive plan...")
    processed_settings = json.loads(json.dumps(settings_deck))

    unit_info = unit_outline.get("unitInformation", {})
    processed_settings['course_id'] = unit_info.get("unitCode",
↳ "UNKNOWN_COURSE")
    processed_settings['unit_name'] = unit_info.get("unitName", "Unknown Unit
↳ Name")

    # --- Apply test overrides IF they are not None ---
    logger.info("Applying overrides if specified...")
    # This block now correctly sets the teaching_flow_id based on the
↳ interactive flag.
    if TEST_OVERRIDE_FLOW_ID is not None:
        processed_settings['teaching_flow_id'] = TEST_OVERRIDE_FLOW_ID
        logger.info(f"OVERRIDE: teaching_flow_id set to
↳ '{TEST_OVERRIDE_FLOW_ID}'")
    else:
        # If no override, use the 'interactive' boolean from the file as the
↳ source of truth.
        is_interactive = processed_settings.get('interactive', False)
        if is_interactive:
            processed_settings['teaching_flow_id'] = 'apply_topic_interactive'
        else:
            processed_settings['teaching_flow_id'] = 'standard_lecture'
        logger.info(f"Loaded from settings: 'interactive' is {is_interactive}.
↳ Set teaching_flow_id to '{processed_settings['teaching_flow_id']}'")

    # The 'interactive' flag is now always consistent with the teaching_flow_id.
    processed_settings['interactive'] = "interactive" in
↳ processed_settings['teaching_flow_id'].lower()

    if TEST_OVERRIDE_SESSIONS_PER_WEEK is not None:
        processed_settings['week_session_setup']['sessions_per_week'] =
↳ TEST_OVERRIDE_SESSIONS_PER_WEEK

```

```

        logger.info(f"OVERRIDE: sessions_per_week set to_
↳{TEST_OVERRIDE_SESSIONS_PER_WEEK}")

    if TEST_OVERRIDE_DISTRIBUTION_STRATEGY is not None:
        processed_settings['week_session_setup']['distribution_strategy'] =_
↳TEST_OVERRIDE_DISTRIBUTION_STRATEGY
        logger.info(f"OVERRIDE: distribution_strategy set to_
↳'{TEST_OVERRIDE_DISTRIBUTION_STRATEGY}'")

    if TEST_OVERRIDE_WEEKS is not None:
        processed_settings['generation_scope']['weeks'] = TEST_OVERRIDE_WEEKS
        logger.info(f"OVERRIDE: generation_scope weeks set to_
↳{TEST_OVERRIDE_WEEKS}")

    # --- DYNAMIC SLIDE BUDGET CALCULATION (Phase 1) ---
    logger.info("Calculating preliminary slide budget based on session time...")

    params = processed_settings.get('parameters_slides', {})
    SLIDES_PER_HOUR = params.get('slides_per_hour', 18)

    duration_hours = processed_settings['week_session_setup'].
↳get('session_time_duration_in_hour', 1.0)
    sessions_per_week = processed_settings['week_session_setup'].
↳get('sessions_per_week', 1)

    logger.info(f" Duration Hours {duration_hours}.")
    logger.info(f" Sessions per week {sessions_per_week}.")

    slides_content_per_session = int(duration_hours * SLIDES_PER_HOUR)
    target_total_slides = slides_content_per_session * sessions_per_week

    processed_settings['slide_count_strategy']['target_total_slides'] =_
↳target_total_slides
    processed_settings['slide_count_strategy']['slides_content_per_session'] =_
↳slides_content_per_session
    logger.info(f" Preliminary weekly content slide target calculated:_
↳# {target_total_slides} slides.")

    # --- Resolve Generation Scope if not overridden ---
    if TEST_OVERRIDE_WEEKS is None and processed_settings.
↳get('generation_scope', {}).get('weeks') == "all":
        num_weeks = len(unit_outline.get('weeklySchedule', []))

```

```

        processed_settings['generation_scope']['weeks'] = list(range(1,
↪num_weeks + 1))

    # --- Save the processed settings to disk ---
    logger.info(f"Saving preliminary processed configuration to:
↪{PROCESSED_SETTINGS_PATH}")
    with open(PROCESSED_SETTINGS_PATH, 'w', encoding='utf-8') as f:
        json.dump(processed_settings, f, indent=2)
    logger.info("File saved successfully.")

    # --- Assemble master config for optional preview ---
    master_config = {
        "processed_settings": processed_settings,
        "unit_outline": unit_outline,
        "book_toc": book_toc,
        # "teaching_flows": teaching_flows
    }

    print_header("Phase 1 Configuration Complete", char="-")
    logger.info("Master configuration object is ready for the Planning Agent.")
    return master_config

```

3 Extract TOC from epub or PDF

```

[3]: # Cell 4: Extract ToC for EACH source + classify (book vs publication)

from ebooklib import epub, ITEM_NAVIGATION
from bs4 import BeautifulSoup
import fitz # PyMuPDF
import json, os, re, urllib.parse
from pathlib import Path
from typing import List, Dict, Tuple, Optional

# -----
# Patterns for doc-type signals
# -----
DOI_RE = re.compile(r"\b10\.\d{4,9}/[-._;()/:A-Z0-9]+\b", re.IGNORECASE) #
↪Crossref regex :contentReference[oaicite:4]{index=4}
ISBN_RE = re.compile(r"\b(?:ISBN(?:-1[03])?:?\s*)(97[89] [-\s]? \d{1,5} [-\s]?
↪\d{1,7} [-\s]? \d{1,7} [-\s]? \d)\b", re.IGNORECASE)

def clean_epub_href(href: str) -> str:
    if not href: return ""
    cleaned_href = href.split('#')[0]
    return urllib.parse.unquote(cleaned_href)

```

```

def parse_navpoint(navpoint: BeautifulSoup, counter: List[int], level: int = 0) → Optional[Dict]:
    title = navpoint.navLabel.text.strip()
    if not title: return None
    content_tag = navpoint.find('content', recursive=False)
    link_filename = clean_epub_href(content_tag['src']) if content_tag else ""
    node = {"level": level, "toc_id": counter[0], "title": title,
    ↪ "link_filename": link_filename, "children": []}
    counter[0] += 1
    for child in navpoint.find_all('navPoint', recursive=False):
        child_node = parse_navpoint(child, counter, level + 1)
        if child_node: node["children"].append(child_node)
    return node

def parse_li(li_element: BeautifulSoup, counter: List[int], level: int = 0) → Optional[Dict]:
    a_tag = li_element.find('a', recursive=False)
    if not a_tag: return None
    title = a_tag.get_text(strip=True)
    if not title: return None
    link_filename = clean_epub_href(a_tag.get('href'))
    node = {"level": level, "toc_id": counter[0], "title": title,
    ↪ "link_filename": link_filename, "children": []}
    counter[0] += 1
    nested_ol = li_element.find('ol', recursive=False)
    if nested_ol:
        for sub_li in nested_ol.find_all('li', recursive=False):
            child_node = parse_li(sub_li, counter, level + 1)
            if child_node: node["children"].append(child_node)
    return node

def extract_epub_toc_and_meta(epub_path: str) → Tuple[List[Dict], Dict]:
    book = epub.read_epub(epub_path)
    toc_data: List[Dict] = []
    id_counter = [0]

    # EPUB metadata (DC)
    meta = {
        "title": (book.get_metadata("DC", "title") or [("", {})])[0][0],
        "creator": (book.get_metadata("DC", "creator") or [("", {})])[0][0],
        "identifier": (book.get_metadata("DC", "identifier") or [("",
    ↪ {})])[0][0],
        "language": (book.get_metadata("DC", "language") or [("", {})])[0][0],
        "publisher": (book.get_metadata("DC", "publisher") or [("", {})])[0][0],
    }

    for nav_item in book.get_items_of_type(ITEM_NAVIGATION):

```



```

soup = BeautifulSoup(nav_item.get_content(), 'xml')
if nav_item.get_name().endswith('.ncx'):
    navmap = soup.find('navMap')
    if navmap:
        for navpoint in navmap.find_all('navPoint', recursive=False):
            node = parse_navpoint(navpoint, id_counter, level=0)
            if node: toc_data.append(node)
    else:
        toc_nav = soup.select_one('nav[epub:type="toc"]')
        if toc_nav:
            top_ol = toc_nav.find('ol', recursive=False)
            if top_ol:
                for li in top_ol.find_all('li', recursive=False):
                    node = parse_li(li, id_counter, level=0)
                    if node: toc_data.append(node)
        if toc_data:
            break

return toc_data, meta

def build_pdf_hierarchy_with_ids(toc_list: List) -> List[Dict]:
    root = []
    parent_stack = {-1: {"children": root}}
    id_counter = [0]
    for level, title, page, *_ in toc_list:
        normalized_level = level - 1
        node = {
            "level": normalized_level,
            "toc_id": id_counter[0],
            "title": (title or "").strip(),
            "page": page,          # 1-based per PyMuPDF docs :
            ↪contentReference[oaicite:5]{index=5}
            "children": []
        }
        id_counter[0] += 1
        parent = parent_stack.get(normalized_level - 1)
        if parent:
            parent["children"].append(node)
        parent_stack[normalized_level] = node
    return root

def pdf_first_pages_text(doc: fitz.Document, n_pages: int = 2) -> str:
    parts = []
    for i in range(min(n_pages, doc.page_count)):
        parts.append(doc.load_page(i).get_text("text") or "")
    return "\n".join(parts)

```

```

def flatten_toc_count(toc_nodes: List[Dict]) -> int:
    if not toc_nodes: return 0
    count = 0
    stack = list(toc_nodes)
    while stack:
        node = stack.pop()
        count += 1
        stack.extend(node.get("children", []))
    return count

def classify_doc_kind(ext: str, page_count: int, toc_nodes: List[Dict], meta: Dict,
    sample_text: str) -> Tuple[str, Dict]:
    signals = {
        "doi": None,
        "isbn": None,
        "has_toc": bool(toc_nodes),
        "toc_entries": flatten_toc_count(toc_nodes),
        "page_count": page_count,
    }

    doi_match = DOI_RE.search(sample_text or "")
    if doi_match:
        signals["doi"] = doi_match.group(0)

    isbn_match = ISBN_RE.search((sample_text or "") + " " + (meta.get("identifier") or ""))
    if isbn_match:
        signals["isbn"] = isbn_match.group(1)

    text_lower = (sample_text or "").lower()
    has_abstract = "abstract" in text_lower
    has_references = "references" in text_lower or "bibliography" in text_lower

    score_pub = 0
    score_book = 0

    if signals["doi"]: score_pub += 3
    if has_abstract: score_pub += 1
    if has_references: score_pub += 1
    if page_count and page_count < 60: score_pub += 1

    if signals["isbn"]: score_book += 3
    if signals["toc_entries"] >= 30: score_book += 2
    if page_count and page_count >= 120: score_book += 2

    doc_kind = "publication" if score_pub > score_book else "book"
    return doc_kind, signals

```

```

def extract_pdf_toc_and_meta(pdf_path: str) -> Tuple[List[Dict], Dict, int,
↳str]:
    doc = fitz.open(pdf_path)
    toc_raw = doc.get_toc() # outlines/bookmarks :contentReference[oaicite:
↳6]{index=6}
    toc_nodes = build_pdf_hierarchy_with_ids(toc_raw) if toc_raw else []
    meta = doc.metadata or {} # PyMuPDF metadata :
↳contentReference[oaicite:7]{index=7}
    page_count = doc.page_count # page_count :
↳contentReference[oaicite:8]{index=8}
    sample_text = pdf_first_pages_text(doc, n_pages=2)
    doc.close()
    return toc_nodes, meta, page_count, sample_text

# -----
# MAIN: loop over SOURCES
# -----
TOC_DIR = Path(TOC_DIR) # ensure Path
TOC_DIR.mkdir(parents=True, exist_ok=True)

enriched_sources = []
for s in SOURCES:
    path = s["path"]
    ext = s["ext"]
    source_id = s["source_id"]

    toc_out = TOC_DIR / f"{source_id}_toc.json"
    meta_out = TOC_DIR / f"{source_id}_meta.json"

    toc_nodes: List[Dict] = []
    meta: Dict = {}
    page_count = 0
    sample_text = ""

    try:
        if ext == ".epub":
            toc_nodes, meta = extract_epub_toc_and_meta(path) # EbookLib
↳metadata :contentReference[oaicite:9]{index=9}
            elif ext == ".pdf":
                toc_nodes, meta, page_count, sample_text =
↳extract_pdf_toc_and_meta(path)
            else:
                raise ValueError(f"Unsupported ext: {ext}")

    # Classification (book vs publication)

```

```

        doc_kind, signals = classify_doc_kind(ext, page_count, toc_nodes, meta,
↪sample_text)

        # Write per-source TOC + meta
        with open(toc_out, "w", encoding="utf-8") as f:
            json.dump(toc_nodes, f, indent=2, ensure_ascii=False)

        with open(meta_out, "w", encoding="utf-8") as f:
            json.dump({"meta": meta, "signals": signals, "doc_kind": doc_kind},
↪f, indent=2, ensure_ascii=False)

        enriched = dict(s)
        enriched.update({
            "toc_json": str(toc_out),
            "meta_json": str(meta_out),
            "doc_kind": doc_kind,
            **signals
        })
        enriched_sources.append(enriched)

        print(f" {doc_kind.upper():11} | {ext:5} | toc={signals['toc_entries']}:
↪4} | pages={page_count:4} | {path}")

    except Exception as e:
        print(f" Failed: {path} :: {e}")
        enriched = dict(s)
        enriched.update({"error": str(e)})
        enriched_sources.append(enriched)

# Update manifest (so downstream chunking/retrieval knows doc_kind + toc path)
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump({"sources": enriched_sources}, f, indent=2, ensure_ascii=False)

print(f"\nManifest updated: {MANIFEST_PATH}")
print(f"ToCs written under: {TOC_DIR}")

```

```

BOOK          | .pdf | toc= 90 | pages= 121 |
inputs/sources/MBAI5004/978-3-031-17040-9.pdf
BOOK          | .pdf | toc= 30 | pages= 533 | inputs/sources/MBAI5004/Ethics
Of Artificial Intelligence -- S_ Matthew Liao -- 1st Edition, 2020 -- Oxford
University Press, Incorporated -- 9780190067403 --
a023d0c0892cc2fe1178a7384fd9d3d9 -- Anna's Archive.pdf
BOOK          | .pdf | toc= 260 | pages= 392 | inputs/sources/MBAI5004/Ethics,
Governance, and Policies in Artificial Intelligence -- Luciano Floridi(eds_) --
Philosophical studies series, v_ 144, Cham, Switzerland, -- 9783030819064 --
3c5a3257f4bc677996586239e0be16e6 -- Anna's Archiv.pdf
PUBLICATION   | .pdf | toc= 0 | pages= 14 |

```

inputs/sources/MBAI5004/ssrn-4845268.pdf

Manifest updated: data_output/MBAI5004/sources_manifest.json

ToCs written under: data_output/MBAI5004/toc

PUBLICATION | .pdf | toc= 0 | pages= 14 |

inputs/sources/MBAI5004/ssrn-4845268.pdf

Manifest updated: data_output/MBAI5004/sources_manifest.json

ToCs written under: data_output/MBAI5004/toc

4 Hirachical DB base on TOC

<https://chatgpt.com/c/WEB:dbf26b2e-f629-46d7-8380-ba6c2a501898>

4.1 Process Books

```
[4]: # =====  
# CELL 5 - MULTI-SOURCE RAG BUILDER (Chroma + stable IDs)  
# =====  
  
import json  
import hashlib  
from pathlib import Path  
from typing import Dict, Any, List, Tuple, Optional  
  
import fitz # PyMuPDF (already used in your Cell 4)  
from bs4 import BeautifulSoup  
from ebooklib import epub, ITEM_DOCUMENT  
  
from langchain_core.documents import Document  
from langchain_community.document_loaders import PyPDFLoader  
from langchain_ollama.embeddings import OllamaEmbeddings  
from langchain_chroma import Chroma  
from langchain.text_splitter import RecursiveCharacterTextSplitter  
  
# -----  
# 0) Paths for incremental state  
# -----  
INDEXED_IDS_PATH = OUTPUT_ROOT / "indexed_chunk_uids.txt"  
PARENTS_JSONL_PATH = OUTPUT_ROOT / "parents_docstore.jsonl" # only used if  
    ↪ USE_PARENT_CHILD  
  
# -----  
# 1) Small utilities  
# -----
```

```

def _sha1(s: str) -> str:
    return hashlib.sha1(s.encode("utf-8", errors="ignore")).hexdigest()

def _stable_chunk_uid(source_id: str, parent_id: str, locator: str, text: str) → str:
    # stable across runs as long as (source, parent, locator, text) are stable
    text_h = _sha1(text.strip()[:5000])[:16] # cap for speed; still
    → stable-enough
    return _sha1(f"{source_id}|{parent_id}|{locator}|{text_h}")[:24]

def _load_manifest(manifest_path: Path) -> List[Dict[str, Any]]:
    data = json.loads(manifest_path.read_text(encoding="utf-8"))
    # Your pipeline sometimes writes {"unit":..., "sources":[...] } and later
    → {"sources":[...] }
    return data["sources"] if "sources" in data else data.get("sources", [])

def _load_json(path: str) -> Any:
    return json.loads(Path(path).read_text(encoding="utf-8"))

def _clean_meta(v: Any) -> Any:
    if isinstance(v, list): return ", ".join(map(str, v))
    if isinstance(v, dict): return json.dumps(v, ensure_ascii=False)
    if isinstance(v, (str, int, float, bool)) or v is None: return v
    return str(v)

# -----
# 2) ToC helpers (flatten with title path + keep toc_id from Cell 4)
# -----
def flatten_toc_with_paths(toc_nodes: List[Dict[str, Any]]) -> List[Dict[str,
→ Any]]:
    flat: List[Dict[str, Any]] = []

    def _rec(nodes: List[Dict[str, Any]], path: List[str]):
        for n in nodes or []:
            title = (n.get("title") or "").strip()
            if not title:
                continue
            toc_id = n.get("toc_id", -1)
            entry = {
                "toc_id": toc_id,
                "titles_path": path + [title],
                "level": n.get("level", 0),
                "page": n.get("page"), # PDF only
                "link_filename": n.get("link_filename"), # EPUB only
            }
            flat.append(entry)

    _rec(toc_nodes, [])

```

```

        _rec(n.get("children", []), path + [title])

    _rec(toc_nodes, [])
    return flat

# -----
# 3) Loaders
# -----
def load_pdf_pages(path: str) -> List[Document]:
    # per-page documents with metadata["page"] 0-indexed
    return PyPDFLoader(path).load()

def load_epub_spine_docs(path: str) -> List[Document]:
    """
    More reliable than matching ToC titles in raw text:
    we extract each spine item text and keep its filename so we can map ToC by
    ↪ link_filename.
    """
    book = epub.read_epub(path)

    # spine is list of (idref, linear)
    spine_idrefs = [idref for (idref, linear) in book.spine if idref and idref !=
    ↪ "nav"]
    docs: List[Document] = []

    for order, idref in enumerate(spine_idrefs):
        item = book.get_item_with_id(idref)
        if not item:
            continue
        name = item.get_name() or ""
        try:
            html = item.get_content().decode("utf-8", errors="ignore")
        except Exception:
            html = str(item.get_content())
        text = BeautifulSoup(html, "html.parser").get_text(separator="\n")
        text = "\n".join([ln.strip() for ln in text.splitlines() if ln.strip()])
        if not text.strip():
            continue
        docs.append(Document(page_content=text, metadata={"epub_item": name,
        ↪ "spine_order": order}))
    return docs

# -----
# 4) Enrichment: attach consistent metadata + ToC mapping
# -----

```

```

def enrich_docs_for_source(source: Dict[str, Any]) -> List[Document]:
    src_path = source["path"]
    ext = source["ext"]
    source_id = source["source_id"]
    work_id = source.get("work_id", "")
    doc_kind = source.get("doc_kind", "unknown")

    toc_nodes = []
    if source.get("toc_json") and Path(source["toc_json"]).exists():
        toc_nodes = _load_json(source["toc_json"])

    flat_toc = flatten_toc_with_paths(toc_nodes)
    # For PDF mapping we need toc entries sorted by page
    flat_toc_pdf = [e for e in flat_toc if e.get("page") is not None]
    flat_toc_pdf.sort(key=lambda x: x["page"])

    if ext == ".pdf":
        pages = load_pdf_pages(src_path)
        enriched: List[Document] = []

        for d in pages:
            page0 = int(d.metadata.get("page", -1))
            page1 = page0 + 1 # ToC pages are 1-based in your Cell 4
            base_meta = {
                "unit": UNIT_CODE,
                "source_id": source_id,
                "work_id": work_id,
                "doc_kind": doc_kind,
                "source_file": Path(src_path).name,
                "page_number": page1,
            }

            # If it's a "book" and has ToC pages, map by "closest toc page <=
            ↪ this page"
            best = None
            if doc_kind == "book" and flat_toc_pdf:
                for e in flat_toc_pdf:
                    if e["page"] <= page1:
                        best = e
                    else:
                        break

            if best:
                for i, t in enumerate(best["titles_path"]):
                    base_meta[f"level_{i+1}_title"] = t
                    base_meta["toc_id"] = best["toc_id"]
            else:

```



```

        base_meta["toc_id"] = -1
        base_meta["level_1_title"] = "Publication / Unstructured"

    cleaned = {k: _clean_meta(v) for k, v in base_meta.items()}
    enriched.append(Document(page_content=d.page_content,
↪metadata=cleaned))

    return enriched

if ext == ".epub":
    docs = load_epub_spine_docs(src_path)

    # Map EPUB by filename (link_filename) → epub_item
    # Compare by basename to handle paths like "Text/ch01.xhtml"
    toc_by_basename: Dict[str, Dict[str, Any]] = {}
    for e in flat_toc:
        lf = e.get("link_filename") or ""
        if lf:
            toc_by_basename[Path(lf).name] = e

    enriched: List[Document] = []
    for d in docs:
        item_name = d.metadata.get("epub_item", "")
        base = Path(item_name).name

        base_meta = {
            "unit": UNIT_CODE,
            "source_id": source_id,
            "work_id": work_id,
            "doc_kind": doc_kind,
            "source_file": Path(src_path).name,
            "epub_item": item_name,
        }

        match = toc_by_basename.get(base)
        if match:
            for i, t in enumerate(match["titles_path"]):
                base_meta[f"level_{i+1}_title"] = t
                base_meta["toc_id"] = match["toc_id"]
            else:
                base_meta["toc_id"] = -1
                base_meta["level_1_title"] = "EPUB / Unmapped"

        cleaned = {k: _clean_meta(v) for k, v in base_meta.items()}
        enriched.append(Document(page_content=d.page_content,
↪metadata=cleaned))

```

```

        return enriched

    # ignore unsupported extensions (your config usually filters these earlier)
    return []

def get_chroma_max_batch(vdb, default=4000) -> int:
    client = vdb._collection._client
    if hasattr(client, "get_max_batch_size"):
        return int(client.get_max_batch_size())
    if hasattr(client, "max_batch_size"):
        return int(client.max_batch_size)
    return default

def add_documents_batched(vdb, docs, ids, safety=0.90):
    max_bs = get_chroma_max_batch(vdb)
    batch_size = max(1, int(max_bs * safety))
    print(f"Chroma max_batch_size={max_bs}, using batch_size={batch_size}")

    for start in range(0, len(docs), batch_size):
        end = start + batch_size
        vdb.add_documents(documents=docs[start:end], ids=ids[start:end])
        print(f"    upserted {end if end < len(docs) else len(docs)} / ␣
↪{len(docs)}")

# -----
# 5) Chunking: normal or parent/child
# -----
def build_chunks(enriched_docs: List[Document], source_id: str) -> ␣
↪Tuple[List[Document], List[Document]]:
    """
    Returns: (child_chunks_for_vectorstore, parent_docs_for_docstore)
    """
    if not enriched_docs:
        return [], []

    if not USE_PARENT_CHILD:
        splitter = RecursiveCharacterTextSplitter(
            chunk_size=CHUNK_SIZE,
            chunk_overlap=CHUNK_OVERLAP,
            length_function=len,
        )
        chunks = splitter.split_documents(enriched_docs)
        out: List[Document] = []
        for c in chunks:

```

```

        locator = f"p{c.metadata.get('page_number', '?')}_toc{c.metadata.
↳get('toc_id', '-1')}"
        uid = _stable_chunk_uid(source_id, parent_id="none",
↳locator=locator, text=c.page_content)
        c.metadata["chunk_uid"] = uid
        out.append(c)
    return out, []

# Parent/child mode:
# 1) group by (toc_id) so parents stay within a section (for books)
groups: Dict[str, Dict[str, Any]] = {}
for d in enriched_docs:
    toc_id = str(d.metadata.get("toc_id", -1))
    key = f"{source_id}::toc::{toc_id}"
    g = groups.setdefault(key, {"texts": [], "meta": dict(d.metadata),
↳"pages": []})
    g["texts"].append(d.page_content)
    if "page_number" in d.metadata:
        g["pages"].append(int(d.metadata["page_number"]))

section_docs: List[Document] = []
for g in groups.values():
    text = "\n\n".join(g["texts"]).strip()
    meta = dict(g["meta"])
    if g["pages"]:
        meta["page_start"] = min(g["pages"])
        meta["page_end"] = max(g["pages"])
    section_docs.append(Document(page_content=text, metadata=meta))

parent_splitter = RecursiveCharacterTextSplitter(
    chunk_size=PARENT_CHUNK_SIZE,
    chunk_overlap=0,
    length_function=len,
)
child_splitter = RecursiveCharacterTextSplitter(
    chunk_size=CHILD_CHUNK_SIZE,
    chunk_overlap=CHILD_CHUNK_OVERLAP,
    length_function=len,
)

parents = parent_splitter.split_documents(section_docs)

child_chunks: List[Document] = []
parent_docs: List[Document] = []
for p_i, p in enumerate(parents):
    locator = f"toc{p.metadata.get('toc_id', '-1')}_p{p.metadata.
↳get('page_start', '?')}-{p.metadata.get('page_end', '?')}_pi{p_i}"

```

```

        parent_id = _sha1(f"{source_id}|{locator}|{_sha1(p.page_content)[:
↳16]}")[:24]
        p.metadata["parent_id"] = parent_id
        p.metadata["parent_locator"] = locator
        parent_docs.append(p)

        kids = child_splitter.split_documents([p])
        for k_i, k in enumerate(kids):
            k.metadata["child_index"] = k_i
            uid = _stable_chunk_uid(source_id, parent_id=parent_id,
↳locator=f"{locator}_ci{k_i}", text=k.page_content)
            k.metadata["chunk_uid"] = uid
            child_chunks.append(k)

        return child_chunks, parent_docs

# -----
# 6) Build / append to Chroma
# -----
sources = _load_manifest(Path(MANIFEST_PATH))
print(f"Loaded {len(sources)} sources from manifest: {MANIFEST_PATH}")

if REBUILD_INDEX and Path(CHROMA_PERSIST_DIR).exists():
    import shutil
    shutil.rmtree(CHROMA_PERSIST_DIR)
    if INDEXED_IDS_PATH.exists():
        INDEXED_IDS_PATH.unlink()
    if PARENTS_JSONL_PATH.exists():
        PARENTS_JSONL_PATH.unlink()
    print("Rebuild requested: wiped existing index + incremental state.")

indexed: set[str] = set()
if INDEXED_IDS_PATH.exists():
    indexed = set([ln.strip() for ln in INDEXED_IDS_PATH.
↳read_text(encoding="utf-8").splitlines() if ln.strip()])

embedding_model = OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA)

vector_db = Chroma(
    persist_directory=str(CHROMA_PERSIST_DIR),
    collection_name=CHROMA_COLLECTION_NAME,
    embedding_function=embedding_model,
)

new_ids: List[str] = []
new_docs: List[Document] = []

```

```

new_parents: List[Document] = []

for s in sources:
    if s.get("error"):
        print(f"SKIP (error) {s.get('path')}: {s['error']}")
        continue

    enriched_docs = enrich_docs_for_source(s)
    child_chunks, parents = build_chunks(enriched_docs,
    ↪source_id=s["source_id"])

    # keep only chunks not indexed yet
    for c in child_chunks:
        uid = c.metadata.get("chunk_uid")
        if not uid or uid in indexed:
            continue
        new_ids.append(uid)
        new_docs.append(c)
        indexed.add(uid)

    new_parents.extend(parents)

print(f"Prepared {len(new_docs)} NEW chunks to index (skipping already-indexed).
    ↪")

if new_docs:
    add_documents_batched(vector_db, new_docs, new_ids)
    INDEXED_IDS_PATH.write_text("\n".join(sorted(indexed)), encoding="utf-8")
    print(f" Added {len(new_docs)} chunks to Chroma: {CHROMA_PERSIST_DIR}")
else:
    print(" Nothing new to index.")

# Persist parent docstore (JSONL) if enabled
if USE_PARENT_CHILD and new_parents:
    with open(PARENTS_JSONL_PATH, "a", encoding="utf-8") as f:
        for p in new_parents:
            rec = {
                "parent_id": p.metadata.get("parent_id"),
                "metadata": p.metadata,
                "page_content": p.page_content,
            }
            f.write(json.dumps(rec, ensure_ascii=False) + "\n")
        print(f" Appended {len(new_parents)} parent docs to: {PARENTS_JSONL_PATH}")

print(f"Total vectors now: {vector_db._collection.count()}")

```

2025-12-28 13:20:02,694 - INFO - Anonymized telemetry enabled. See <https://docs.trychroma.com/telemetry> for more information.

Loaded 4 sources from manifest: data_output/MBAI5004/sources_manifest.json
Prepared 8780 NEW chunks to index (skipping already-indexed).
Chroma max_batch_size=5461, using batch_size=4914

2025-12-28 13:21:17,382 - INFO - HTTP Request: POST
<http://127.0.0.1:11434/api/embed> "HTTP/1.1 200 OK"

upserted 4914 / 8780

2025-12-28 13:22:01,874 - INFO - HTTP Request: POST
<http://127.0.0.1:11434/api/embed> "HTTP/1.1 200 OK"

upserted 8780 / 8780

Added 8780 chunks to Chroma: data_output/MBAI5004/chroma_db
Appended 1997 parent docs to: data_output/MBAI5004/parents_docstore.jsonl
Total vectors now: 8780